

Creating a Language for Writing Real-Time Applications for the Internet of Things

ROBERT KROOK, Chalmers University of Technology, Sweden

BO JOEL SVENSSON, Chalmers University of Technology, Sweden

JOHN HUI, Columbia University, USA

STEPHEN A. EDWARDS, Columbia University, USA

KOEN CLAESSEN, Chalmers University of Technology, Sweden

We describe the development of a new programming language Scoria and its compiler. Scoria is a high-level reactive real-time language based on the sparse synchronous model (SSM), designed to produce time- and power-efficient low-level C code that can run on small IoT devices. While the compiler is not yet in a state where it is meaningful to measure power usage, we carefully profile the timing behaviour and identify bottlenecks that can improve performance. The language and compiler are implemented as an Embedded Domain-Specific Language (EDSL) on top of Haskell.

Additional Key Words and Phrases: Real-time, IoT, Compilers, Embedded Domain-Specific Languages

1 INTRODUCTION

Devices for the Internet of Things (IoT) typically contain hardware for sensors, actuators, and wireless communication, and often need to run on batteries whose life expectancy is a couple of years. These peripherals and resources need to be carefully managed in order to maximize lifetime and reliability. The devices are typically programmed in C because of their low-level nature, timing, and power requirements. They are coded in a *reactive* style as the software needs to react to external events, such as a triggered sensor. Such programs are often written to register a myriad of callback functions to handle these events and communicate with other parts of the program. The resulting asynchronous code is often error-prone and hard to maintain, a situation often dubbed “callback hell.”

Our language, Scoria, eschews callbacks in favor of lightweight threads that can block on external and internal events and provides precise control over the time at which code executes. We embed [14] Scoria in Haskell—which explains its syntactic idiosyncrasies—to enable rapid development of its compiler, which generates C code that uses our runtime system (RTS) to interface with hardware timers and other peripherals.

Figure 1 is a simple Scoria program: a remote-controlled square wave signal generator whose frequency can be adjusted by Bluetooth Low Energy (BLE) radio

Authors’ addresses: Robert Krook, krookr@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden; Bo Joel Svensson, joels@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden; John Hui, j-hui@cs.columbia.edu, Columbia University, New York, USA; Stephen A. Edwards, sedwards@cs.columbia.edu, Columbia University, New York, USA; Koen Claessen, koen@chalmers.se, Chalmers University of Technology, Gothenburg, Sweden.

```

1  sigGen :: (?out0 :: Ref GPIO) => Ref Word64 -> SSM ()
2  sigGen hperiod = routine $ while true (do
3    after (ns (deref hperiod)) ?out0 (not' (deref ?out0))
4    wait ?out0)
5
6  remoteControl :: (?ble :: BLE) => Ref Word64 -> SSM ()
7  remoteControl hperiod = routine $ do
8    enableScan ?ble
9    while true (do
10     wait (scanref ?ble)
11     if deref (scanref ?ble) ==. 0
12       then hperiod <~ deref hperiod * 2
13       else hperiod <~ max' (deref hperiod /. 2) 1)
14
15  entry :: (?ble :: BLE, ?out0 :: Ref GPIO) => SSM ()
16  entry = routine $ do
17    hperiod <- var (time2ns (secs 1))
18    fork [sigGen hperiod, remoteControl hperiod]
19

```

Fig. 1. A remote-control signal generator in Scoria. `sigGen` produces a square wave on the `out0` GPIO pin that cycles every two `hperiod` nanoseconds. When a Bluetooth packet arrives, `remoteControl` doubles or halves the period.

packets. The `entry` routine (lines 15–18) expects a BLE receiver and GPIO pin in the environment (line 15), creates a shared variable `hperiod`, and runs `sigGen` and `remoteControl` concurrently (line 18).

The `sigGen` routine (lines 1–4) generates a precisely timed square wave on the GPIO pin by scheduling future output events and blocking on them. It takes a reference to the `hperiod` variable and enters an infinite loop that schedules a future toggle update to the GPIO pin (line 3), then blocks until the update occurs (line 4). The delay between toggle events is exactly the *model time* set by the `hperiod` variable, regardless of the loop’s execution time—Scoria’s model time only advances at blocking statements like `wait`. The `after` directive schedules the update according to model time.

The `remoteControl` routine (lines 6–13) waits for a BLE packet (line 10), and doubles or halves `hperiod` in response (lines 12 and 13). The `enableScan` call (line 8) enables the Bluetooth receiver, which then generates events on the variable given by `scanref ?ble`.

Figure 2 shows code that runs at compile time to initialize the signal generator program’s environment. References to a GPIO pin and the BLE controller are obtained in lines 3 and 4, and added to the environment (indicated by the question marks) in lines 5 and 6. In lines 7–9, the SSM ready queue is populated with the `entry` routine and I/O handlers for the GPIO pin and BLE. I/O handlers are described in Section 4.

This small example illustrates a key feature of Scoria needed by most IoT applications: concurrent programming with lightweight threads that can block on events. In C, it is easy to write a program that waits for an event from a single source by entering a polling loop, but waiting for multiple events is difficult. A central event-handler loop works but discards control state between events; the programmer must explicitly maintain any state, i.e., what to do next.

Multiple events are typically handled by OS-provided threads, which allows the programmer to write sequential code that only blocks on single events, but IoT devices often do not have the memory to provide a separate stack per thread. Protothreads [11] provides an extremely lightweight thread package in the form of C macros that only require storage for one additional program counter per thread, but the onus remains on the programmer to store all other state globally.

The Sparse Synchronous Model (SSM) of Edwards & Hui [12] integrates this blocking-thread concurrency model with a model of time that results in programs with deterministic functional and temporal behavior. A program running under SSM behaves like a discrete-event simulation: time is divided into instants in which the program executes. At any time, the program only knows what instant it is in and may schedule variable updates for a future instant. The SSM scheduler does its best to keep model time synchronized with real time, but the behaviour of the program is not affected by any deviation. Thread execution within a single instant is totally ordered to eliminate races and guarantee deterministic behavior.

Built atop SSM, Scoria is a programming language designed to simplify prototyping new language features and experiments with language design. Our compiler implements concurrency with C functions that store their state—control state and all local variables—in heap-resident activation records, instead of on the stack. Such code is like what a programmer might write with Protothreads, but Scoria provides more familiar abstractions such as local variables and blocking `wait` statements.

This paper describes the current status of Scoria and the choices we made during its development:

- We implement Scoria as an embedded domain-specific language (EDSL) in Haskell. It is mature enough that we can write programs that use I/O peripherals and run them on SoC-boards, but more work is needed before we can write complicated examples such as mesh networks.
- Programs in Scoria do not have to explicitly manage memory; the compiler and RTS allocate and deallocate memory, sparing us from a garbage collector.
- Scoria’s language interface for interacting with I/O peripherals is the same as for interacting with regular Scoria references. Our RTS provides bindings for Zephyr OS [23].
- We test Scoria’s compiler with QuickCheck [9], a property-based Haskell tester. We check for memory leaks and errors; compare the behaviour of compiled programs with an interpreter; and our domain-specific shrinker reduces errant programs a minimal test case.

```

1  compiler :: Compile ()
2  compiler = do
3      (gpio0, gpiohandler) <- output 0
4      (ble, _, scanhandler) <- enableBasicBLE
5      let ?out0 = gpio0
6          ?ble = ble
7      schedule scanhandler
8      schedule entry
9      schedule gpiohandler
10

```

Fig. 2. The top-level of the signal generator program in Figure 1. This requests a GPIO pin and a BLE radio handler, then runs the entry routine along with I/O handlers.

```

1  #include <zephyr/zephyr.h>
2  #include <zephyr/drivers/gpio.h>
3  static const struct gpio_dt_spec led =
4      GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
5  void main(void) {
6      int ret;
7      if (!device_is_ready(led.port)) return;
8      ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
9      if (ret < 0) return;
10     while (1) {
11         ret = gpio_pin_toggle_dt(&led);
12         if (ret < 0) return;
13         k_msleep(1000); // sleep for 1000 ms
14     }
15 }

```

Fig. 3. The “blinky” example distributed with Zephyr RTOS, which purports to generate a signal with a 2 s period.

- We evaluate the performance of Scoria’s runtime system and generated code by stress testing it on real hardware. We precisely measure latencies to identify bottlenecks.

2 MOTIVATION

To motivate real-time programming languages like Scoria, consider the “blinky” application commonly used as “hello world” in embedded systems programming. This toggles an LED at a specified frequency; Figure 3 shows the sample distributed with the Zephyr real-time operating systems (RTOS).

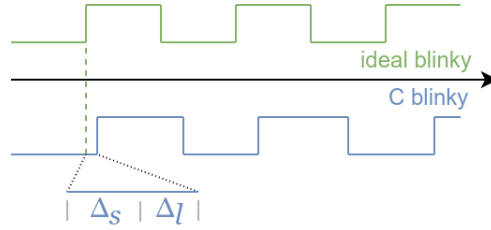


Fig. 4. The C blinky program of Figure 3 drifts due to latency from the RTOS (Δ_s) and the user program (Δ_l).

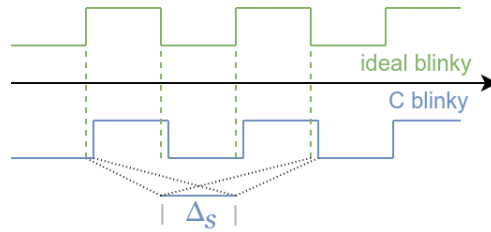


Fig. 5. Blinky without drift.

While blinky (written in C) is adequate for showcasing the basic I/O and timing facilities of an embedded programming framework, it misrepresents its timing behavior: blinky does not actually blink once every two seconds! This discrepancy is due to two sources of latency: the program spends some time each loop iteration to toggle the LED, between invocations of `k_msleep` and once the timer set by `k_msleep` has expired, Zephyr takes some time to reschedule and resume the blinky program. If the time toggling the LED is Δ_l , and the time spent rescheduling the blinky program is Δ_s , then each loop iteration will take $1000 + \Delta_l + \Delta_s$ ms rather than just 1000 ms. Over time, the latency will accumulate into *drift* as the program continues to drag behind its ideal behavior (Figure 4).

To compensate for this latency and eliminate drift, the blinky program should only sleep for as long as it needs to keep up with the ideal timing behavior. To do this, our “corrected” implementation (Listing 2) maintains a *logical* clock of how much time *should* have elapsed, and shortens its sleep time according to the difference between that logical time and the *actual* time. The resulting temporal behavior, shown in Figure 5, does not suffer from drift, only incurring a stable phase error of Δ_s .

However, this technique comes at the cost of complexity: the implementation shown in Listing 2 is 45 LOC, and is deeply entwined with Zephyr’s API. Though certain configuration code may be simplified or omitted, the program’s control flow and timing logic is obfuscated by the need to respond to the timer via interrupt handlers. Furthermore, this technique alone does not scale well beyond sequential

```

1  // include and defines
2  // declaration of button0, button1, led0 \& led1
3
4  uint32_t last = -1;
5  void button_cb(const struct device *dev,
6                struct gpio_callback *cb, uint32_t pins) {
7      if (last == button0_pins && pins == button1_pins)
8          gpio_pin_set_dt(&led0, 1);
9      else if (last == button1_pins && pins == button0_pins)
10         gpio_pin_set_dt(&led1, 1);
11     else {
12         gpio_pin_set_dt(&led0, 0);
13         gpio_pin_set_dt(&led1, 0);
14     }
15     last = pins;
16 }
17
18 // callback initialization and registration

```

Fig. 6. While the program above is semantically simple, the implementation has to install interrupt handlers. Knowledge of which events have occurred must be lifted to the global scope, which must then be consulted before an LED can be lit.

```

1
2  seq :: Ref SW -> Ref SW -> Ref GPIO -> Ref GPIO -> SSM ()
3  seq b0 b1 l0 l1 = routine $ do
4      wait (b0, b1)
5      if changed b0 -- was b0 the button that was pressed?
6          then do wait b1
7                  l0 <~ ON -- we know the sequence was b0, b1
8      else do wait b0
9              l1 <~ ON -- we know the sequence was b1, b0

```

Fig. 7. This Scoria routine implements the same behaviour as Figure 6, but each branch knows the source of its event and which LED to light without consulting a global “last” variable.

programs, since any number of concurrent threads may need to synchronize their use of a finite number of hardware timers.

Scoria overcomes this complexity by incorporating the notion of logical time into its concurrent programming model. In doing so, Scoria allows the programmer to express the *intended* timing behavior, abstracting over platform-specific timer APIs. Scoria relies on its runtime to approximate the intended behavior to the extent possible by making use of available and supported timing capabilities. For comparison, Figure 1 shows blinky in Scoria, which is only 15 LOC.

```

1  routine                                -- Define a Scoria routine body
2  var      :: Exp a -> SSM (Ref a)      -- New local var
3  deref    :: Ref a -> Exp a            -- Access value
4  (<~)     :: Ref a -> Exp a -> SSM ()  -- Assignment
5  after    :: Time -> Ref a -> Exp a -> SSM () -- Delayed assignment
6  changed  :: Ref a -> Exp Bool         -- Was written?
7  wait     :: Waitable a => a -> SSM ()  -- Block on write
8  if-then-else :: Exp Bool -> SSM () -> SSM () -> SSM () -- Conditional
9  while     :: Exp Bool -> SSM () -> SSM () -- Loop
10 fork     :: [SSM ()] -> SSM ()        -- Concurrency

```

Fig. 8. The Core API of Scoria.

Another example that illustrates how using interrupt handlers can complicate code unnecessarily is shown in Figure 6. It is an interrupt handler of a simple program that lights up one of two LEDs depending on which sequence of presses from two different buttons is registered. This program needs to remember which button was pressed previously, so that when the next press arrives it can inspect the complete sequence and decide which LED to light up. In order to remember the previous press, the program needs to use global variables, opening itself up to the possibility of complicated control flow where global variables are modified by several concurrently running threads.

The Scoria version of this does not rely on global variables; it knows which events happened based on which code is executing and thus uses *local* information rather than global.

3 OVERVIEW OF SCORIA

Scoria is an embedded domain-specific language for IoT applications. It is domain-specific in that it is tailored to the demands of IoT applications through its facilities for concurrency and timing control. This makes the language easier to learn, compile, and optimize at the expense of making programming tasks outside its scope more difficult. It is embedded in a host language, Haskell, which allows us to benefit from the existing Haskell tools and compilers while presenting a convenient, new abstraction for the user. Haskell provides e.g the parser and a type checker, and we use host-level evaluation as a kind of macro system (see Section 5). Such an approach enabled us to develop Scoria much more rapidly, making it easy to experiment with new language features.

We modeled Scoria on the proposal by Edwards & Hui [12], with adaptations to embed it in Haskell.

Figure 8 lists the core API of Scoria. These may be thought of as Scoria’s primitives, but each are actually Haskell functions composed using Haskell’s *do*-notation. Values that are embedded (Scoria values) are of type `Exp a`, while values that belong to the host language are of type `a`. Host-level values are evaluated at Scoria compile-time.

The `routine` keyword introduces the body of a Scoria procedure. The Scoria compiler simply executes any Haskell code outside a `routine` block. See Section 5 for details.

A reference to a variable is created with the `var` keyword, can be assigned a new value with the `<~` (“assign”) operator, and can be read with `deref`. The `changed` function returns true if a given variable (reference) has been written in the current instant, e.g., by an assignment in a concurrent thread.

The `after` operation schedules an assignment to a reference in a future instant. Its first argument is the time delay (relative to the current instant) before performing the assignment, and the other two arguments are the variable to update and the future value it should receive (which is evaluated immediately).

The `wait` statement blocks until at least one of the given variables (references) have been written, either by an immediate assignment `<~` or a scheduled assignment (`after`). `wait` takes either a single reference or tuples of references of varying sizes. Haskell’s *type classes*[13] allow us to overload the operator to accept either.

`if-then-else` and `while` are the usual conditional and looping constructs. The code in their branches may block or call functions that do.

Lastly, `fork` is a parallel function call construct that spawns concurrently running child processes and blocks until all children have terminated (and not just blocked). The order of children is significant: it prescribes the order in which the children run in any instant, enforcing deterministic concurrency with a total execution order. In particular, an earlier child may write to a shared variable that a later child may read in the same instant, but not vice-versa (earlier children may only read data from a later child that was assigned in an earlier instant or in a scheduled update).

As shown in Figure 2, Scoria also provides a `Compile` monad that runs code at compile time. This is used to perform static initialization of e.g peripherals, and make them accessible to the rest of the Scoria program. As an example, a user might want to create a global variable `gv` and supply a reference to it to the `main` routine. This can be done as follows:

```

1  program :: Compile ()
2  program = do
3    gv <- global @Word64
4    let ?gv = gv
5    schedule main
6
7  main :: (?gv :: Ref Word64) => SSM ()
8  main = routine $ ?gv <~ 5

```

To start the Scoria program, at least one routine must be added to the ready queue with the `schedule` directive, which populates the ready queue when the program begins. Each successive invocation of `schedule` adds another routine at a lower priority level: those scheduled first will run first in any instant, much like a `fork`.

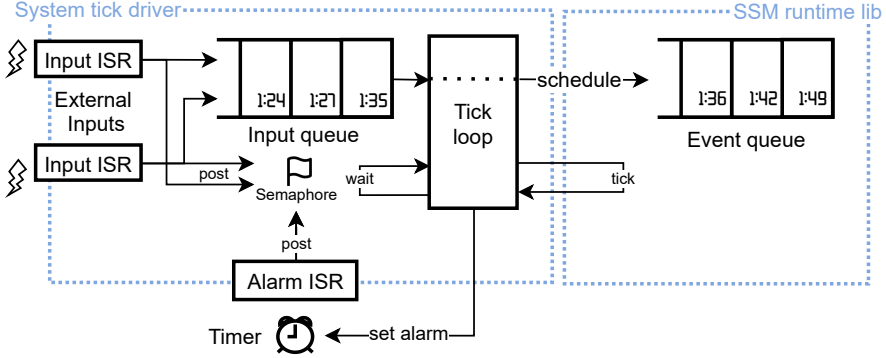


Fig. 9. The SSM runtime’s input handling architecture.

Code generation from Scoria. Edwards & Hui [12] provide an SSM RTS coded in C. We have written a code generator that takes the abstract syntax of a Scoria program and produces C code that calls this RTS. The generated C code is platform agnostic; additional platform-specific code is necessary, e.g., to set a future wake-up alarm. We have written such bindings for Zephyr OS [23] and for something we call the trace platform, which we describe in Section 6.

4 I/O IN SCORIA

A Scoria program communicates with its environment through variables. The Scoria runtime collects input events from interrupt service routines (ISRs)—effectively the callback functions of traditional asynchronous C implementations—and delivers them to the SSM event queue. Writes to output variables are observed by special I/O handler routines that typically call external C functions to perform the output.

Handling External Inputs and Time. Figure 9 shows the structure of the Scoria runtime that handles input events, manages the system timer, and runs the system in each instant. The main tick loop (Figure 11) retrieves events enqueued by ISRs (Figure 10), calls the SSM runtime’s `tick()` of Edwards and Hui [12] to run the system (i.e., the code generated by the Scoria compiler) for a single instant, sets an alarm to wake it up at the next scheduled event, and then sleeps until it is awakened by the alarm or input from a peripheral.

Input events are gathered by ISRs and ultimately appear as scheduled variable updates to the Scoria program, but this process operates in several steps for safety and efficiency.

The Scoria runtime maintains two distinct event queues to avoid excessive synchronization costs. The input queue is loaded asynchronously by ISRs and emptied by the main tick loop, but it is a simple ring buffer that is easy to make thread-safe. By contrast, the main event queue is a priority queue to which events are added

```

1 void input_isr(device *dev) {
2     int key = irq_lock();
3     ssm_time_t input_time = timer_read();
4     ssm_input_event *input = ssm_input_alloc();
5     irq_unlock(key);
6     if (input == NULL) // No space in input queue
7         return;
8     input->time = input_time;
9     input->data = ssm_input_read(dev); // Device-specific
10    input->ref = ssm_input_binding(dev);
11    ssm_input_commit(); // Enqueue input queue
12    sem_post(ssm_sem); // Wake up tick thread
13 }

```

Fig. 10. The structure of an input interrupt handler, which enqueues an event in the input queue. Each peripheral has its own device-specific handler.

and sometimes removed out-of-order by both the tick loop and the various concurrent routines. However, because these run synchronously, the event queue is never accessed asynchronously and need not be thread-safe.

There are several reasons why we need two queues instead of just one. The synchronous hypothesis states that computations are instantaneous, meaning that an invocation of `tick` returns instantly. In reality this is not the case, and physical time will progress. If an external event is delivered to the system as soon as it arrives, `tick()` would look into the future, determinism would break, and we would be susceptible to data races. Furthermore, without time stamping external events, a second external event might arrive and overwrite the previous event before the system has had a chance to observe the first one, leading to events essentially being dropped. The SSM runtime allows only one outstanding event per reference at a single time.

Figure 10 shows an input interrupt handler. It first records the current system time, then attempts to enqueue an event with that timestamp and any new data from the peripheral in the input queue: a ring buffer large enough to accommodate a modest backlog of input events before having to drop or overwrite events. Enqueue and dequeue operations are performed in-place via an allocate/commit and peek/release protocol to minimize copying.

In an ISR, recording the current system time as quickly as possible and allocating space in the input queue is critical to ensure enqueued input events appear with nondecreasing timestamps. Other higher-priority interrupts may occur while processing a lower-priority interrupt, but only after the timestamp of the lower priority interrupt has been captured.

Figure 11 shows the main tick loop, which waits on a OS-provided semaphore (`ssm_sem`) that is posted by an interrupt service routine from either a peripheral or the system clock. At each iteration of the tick loop, the runtime checks the input

```

1  struct semaphore ssm_sem;
2  void alarm_isr(void) { sem_post(&ssm_sem); }
3  void ssm_tick_loop(void) {
4      ssm_init();
5      ssm_tick();
6      for (;;) {
7          real_time = timer_read();
8          model_time = ssm_next_time();
9          ssm_input_event *input = ssm_input_peek();
10         if (input && input->time <= model_time) {
11             ssm_schedule_input(input); // Add to event queue
12             ssm_input_release(); // Remove from input queue
13         } else if (model_time <= real_time) {
14             ssm_tick(); // Run system; advance model time
15         } else if (model_time == NO_EVENT_SCHEDULED) {
16             sem_wait(ssm_sem); // Wait for input
17         } else {
18             if (timer_set_alarm(model_time, alarm_isr))
19                 continue; // Alarm already expired
20             sem_wait(ssm_sem); // Wait for alarm or input
21             timer_cancel_alarm();
22             sem_reset(ssm_sem);
23         }
24     }
25 }

```

Fig. 11. The Scoria runtime’s main tick loop, which gathers events from the input queue, runs `qdetick()` to advance SSM model time, then sleeps until the next scheduled event or an interrupt.

queue for events to schedule in the SSM event queue. Events in the input queue will be in increasing order since we assume the system timer advances monotonically. However, if the logical time of the SSM program is running behind physical time when some external input is received—`input->time > model_time`—or if there is no pending input event, the runtime executes the SSM program by calling `ssm_tick()`. Finally, if there are neither input events to process, nor internal events ready to execute, the main thread goes to sleep, blocking on a semaphore until either its timer expires or some fresh input appears.

Handling External Outputs. Just as with inputs, output peripherals are bound to regular Scoria references; writes to those references are emitted to the environment as output. Under the hood, those writes eventually trigger a call to some system-provided C output function that actualizes the output.

One of the design considerations we faced when implementing outputs was deciding exactly when in an instant the system output function should be invoked. The SSM model says all the code of an instant executes in zero time, but in practice any execution takes time and the code for an instant is effectively spread out over a finite time interval. If an output function is invoked too early, subsequent updates to the output variable in the same instant might be ignored, whereas if it is invoked too late, there will be output latency and perhaps even jitter arising from the execution speed of unrelated computation. For instance, consider the following three concurrent processes, shown in order of decreasing priority:

```

1  w1 :: Ref Button -> Ref LED -> SSM ()
2  w1 button led = routine $ do
3      while true (do
4          wait button
5          led <~ true)
6
7  w2 :: Ref Button -> Ref LED -> SSM ()
8  w2 button led = routine $ do
9      while true (do
10         wait button
11         led <~ false)
12
13  r :: Ref Button -> SSM ()
14  r button led = routine $ do
15      while true (do
16          wait button
17          -- some expensive computation
18      )

```

If the LED is lit immediately when it is first assigned—after the assignment of `true` by `w1`—then the assignment of `false` by `w2` will be missed, leading to a stale value being emitted. But if the LED is lit as late as possible—at the end of the instant—the system must wait until `r` has completed executing, incurring some delay. Depending on the particular application, either one of these outcomes may be unacceptable.

Our design leaves it to the user to decide when the physical output should take place by encapsulating it within an *output handler* process that the user must schedule. This handler is obtained when the user asks for the output device. If a process performs an immediate assignment to a reference, that assignment will only be visible to lower-priority processes. Note that delayed assignments do not have this behaviour; they always wake up every waiting process. For instance, if the user would like to schedule the handler after any possible assignments by `w1` and `w2`, but before `r`, the user may specify:

```

1  initProgram :: Compile ()

```

```

2  initProgram = do
3    button <- input 0
4    (led, handler) <- output 0
5
6    schedule $ w1 button led
7    schedule $ w2 button led
8    schedule $ handler
9    schedule $ r button led

```

While this precludes `r` from performing any effectful *instantaneous* assignments, it still accommodates effectful *delayed* assignments by `r`.

This gives the developer some control over two sources of jitter. The first is due to varying computation time—if an output handler is scheduled to run after a higher priority process in the same instant, then the actual output time will depend on the speed of the first process. This source of jitter can be eliminated by scheduling the output handler so it runs at the beginning of the instant at the expense of any instantaneous assignments—it is up to the user to balance the reliability versus the expressiveness of their application.

Discrepancies in input response times is a second source of jitter. The runtime system is able to handle inputs slightly faster when it is already awake than when it is asleep. If output is to be instantaneously triggered by some input, then the input-to-output latency will differ depending on whether an input is received when the system is asleep or awake.

5 LEVERAGING THE HOST LANGUAGE

Scoria is implemented as a Haskell library that exposes an embedded language interface which allows us to reuse Haskell’s parser and type checker, and otherwise take advantage of Haskell language features for rapid language development. In this section, we describe some of the embedding techniques we use to ensure that our language interface is robust, expressive, and easy to use.

Type Checking. Function Application in Haskell is checked to guarantee the type safety of the application. The value `routine` body has the same type as `body`, so Scoria procedures still look like ordinary Haskell functions and are invoked by performing normal Haskell function application, with the effect that the Haskell compiler will type check Scoria procedure invocations for us, for free. This kind of type checking happens in every corner of the language API, making it harder to construct illegal programs.

Providing New Abstractions. The host language of an embedded language can be used as a macro system, enabling effortless experimentation with language design. As an example, consider the `delay` statement, which blocks for a fixed period of time. This operation is not primitive to Scoria or SSM, but the behaviour can be recovered using existing primitives.

```

1  delay :: Exp Time -> SSM ()
2  delay x = do
3    sync <- var event
4    after x sync event
5    wait sync

```

Because this Haskell function is not decorated with the `routine` keyword, it is not treated as a callable Scoria routine. Instead, when a program invokes `delay`, Scoria will inline `delay`'s body.

Yet this implementation has the downside of introducing a new local variable `sync` in the activation record of its call site each time `delay` is invoked. An alternative implementation which turns `delay` into a Scoria procedure might look like:

```

1  delay' :: Exp Time -> SSM ()
2  delay' x = fork [ delayRoutine x ]
3
4  delayRoutine :: Exp Time -> SSM ()
5  delayRoutine x = routine $ delay x

```

When a program invokes `delay'`, the inlined statement will instead be a `fork` statement that calls the auxiliary `delayRoutine` routine.

Plugin for Source-to-Source Transformations. Scoria uses the `routine` keyword to distinguish between a Haskell function and a Scoria procedure. This distinction is necessary because in the latter case, the compiler must construct AST nodes corresponding to the routine definition, rather than directly evaluating its body. It is this indirection that allows Scoria programs to perform recursion without endlessly inlining the routine body.

Yet routine definitions cannot be trivially embedded in Haskell, because it requires functions to reflect on their name and the names of their arguments. Case in point: an early iteration of our compiler used `make_routine` to construct routines, which expects additional arguments conveying the names of the routine and its arguments. This led to an awkward language interface with plenty of syntactic redundancy:

```

1  f = make_routine "f" ["x", "y"] $ \x y -> body

```

To overcome this burden, Scoria leverages plugin support from the Haskell compiler, GHC, to infer the arguments for `make_routine`. Our plugin looks for invocations of `routine` and replaces it with an invocation of `make_routine` with the necessary source information. For example, this source-to-source transformation produces the above boilerplate from the following:

```

1  f x y = routine body

```

6 TESTING

Interpretation. Following Edwards & Hui [12], we implemented an interpreter in Haskell that steps through instants of model time but does not attempt to synchronize

Bug location	Number of bugs
RTS	1
C generator	8
Interpreter	11

Table 1. Breakdown of the bugs we identified developing of our compiler.

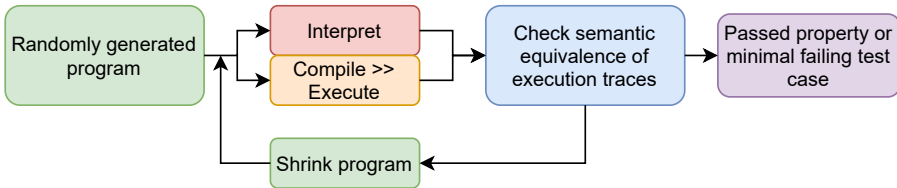


Fig. 12. Overview of how the second property is tested, asserting the semantic behaviour of SSM programs.

with wall-clock time. We use our interpreter as a reference to validate our compiler’s output; see below.

Testing. To debug the Scoria compiler, we used QuickCheck [9] (QC), a property-based testing framework that generates *random* values and tests them against a property. If the property is falsified, QC shrinks the test to find a smaller failing case. The QC-generated random programs are typically too large to diagnose without shrinking.

We test two properties: the first is that generated C code compiles and runs without memory errors or leaks, according to Valgrind. This helped us find a bug related to our event queue: when a variable was deallocated, scheduled events were not being unscheduled and the RTS would attempt to perform updates using these stale pointers, corrupting the program state. We fixed this bug by ensuring that all routines unschedule outstanding updates to local variables before returning.

We also test semantic correctness of the generated program as shown in Figure 12. When testing, our compiler adds print statements to the generated code to produce an event trace, which we compare to one from the reference interpreter. This property helped us identify C generation problems. Most notably, we initially implemented Scoria signed integers with C’s signed integers. Our tester found cases that branched on overflowing arithmetic to produce different behaviour because of the undefined overflow behavior of C’s integers. We fixed this by switching to C’s better-defined unsigned integers.

We documented 20 bugs, mostly semantic errors (Table 1).

```

1  freqCount :: Ref GPIO -> SSM ()
2  freqCount sw = routine $ do
3      count <- var $ u32 0
4      gate <- var event
5      after (secs 1) gate event
6      while true (do
7          if changed gate
8              then do
9                  -- Print count to console
10                 if changed sw
11                     then count <~ 1
12                     else count <~ 0
13                 after (secs 1) gate event
14                 wait gate -- Sleep for 1 sec
15                 after (secs 1) gate event
16             else
17                 count <~ deref count + 1
18             wait (gate, sw))
19
20  freqCountProg :: Compile ()
21  freqCountProg = input 0 >>= schedule . freqCount

```

Fig. 13. A frequency counter program.

7 EXPERIMENTAL RESULTS

We test the performance of our Zephyr-based runtime system by subjecting Scoria programs to varying loads. All experiments are performed using a Nordic Semi NRF52840-DK board with a 64 MHz Cortex-M4 processor, 256 kB RAM, 1 MB flash, BLE support, and a 16 MHz crystal time base. We apply loads with a pulse generator driving a GPIO input pin and measure the results with an oscilloscope on an output pin.

Frequency Counter. To assess our implementation’s resilience to high input load, we test a small Scoria frequency counter, shown in Figure 13. This program measures the frequency of button presses by counting the number of input events each second. The reported count is double the frequency, corresponding to the two input events of a single button press. For benchmarking purposes, we insert a print statement in the generated code to report the frequency; the program alternates between counting and reporting to ensure that the overhead of reporting the frequency does not interfere with the frequency counting at high loads.

The reference `sw` is bound to `input 0` on the NRF52840-DK; we connect a function generator to the corresponding GPIO pin and generate pulses at various frequencies. We found that our frequency counter is capable of measuring up events of up to


```

1  struct semaphore sem;
2  void alarm_isr(void) { sem_post(&sem); }
3
4  uint32_t count = 0;
5  // Executed each time button is pressed or released
6  void input_isr(void) { count++; }
7
8  void main_loop(void) {
9      for (time_t time = timer_read(); true; count = 0) {
10         timer_set_alarm((time += secs(1)), alarm_isr);
11         sem_wait(&sem);
12         printk("frequency * 2: %u Hz\r\n", count);
13         timer_set_alarm((time += secs(1)), alarm_isr);
14         sem_wait(&sem);
15     }
16 }

```

Fig. 14. Frequency counter program implemented in C.

29 kHz (input frequency of 14.5 kHz, each rising or dropping edge creates one event), with error within 2 Hz. Beyond this, the input queue fills up faster than inputs are consumed, leading to events being dropped and the program thrashing. The frequency counter is able to recover from thrashing after we lower the input frequency below 14 kHz.

To measure the overhead of SSM event scheduling, we compare our Scoria frequency counter to the functionally equivalent manually coded C in Figure 14. The C implementation omits the SSM scheduler and directly manages synchronous control flow in `main_loop` using the same semaphore API provided by Zephyr OS. We also forego the input queue and directly increment count in the input ISR.

We find that this optimized version can accurately measure input frequencies of up to 34 kHz with 5 Hz of error, but is more verbose and error-prone, and less expressive and flexible.

Button-to-blink. To better understand the system sans synchronous logic, we first evaluate the “button-to-blink” program in Figure 15. This lights an LED when a button is pressed and turns it off when released. We schedule the LED handler after the main b2b program so that this instantaneous assignment is written to the LED at the end of the instant.

The instantaneous assignment in b2b incurs a practically avoidable but theoretically significant amount of latency, which we call the *at-rest input latency*, δ_r . This value represents the time it takes for the system to wake *from sleep* and respond to external input. From profiling, we find that δ_r for our NRF52840-DK is approximately 60 μ s.

```

1  b2b :: (?sw :: Ref Switch, ?led :: Ref GPIO) => SSM ()
2  b2b = routine $ while true $ do
3      wait ?sw
4      ?led <~ deref ?sw
5
6  b2bc :: Compile ()
7  b2bc = do
8      sw <- input 0
9      (led, handler) <- output 0
10     let ?sw = sw
11         ?led = led
12     schedule b2b
13     schedule handler

```

Fig. 15. Button-to-blink in Scoria; generated C in Listing 9

Frequency (kHz)	Latency (μ s)	Jitter (μ s)
2	60.0	0.7
4	60.6	0.7
6	49.7	0.5
8	52.1	13.1
10	47.3	34.0
12	45.6	24.0

Table 2. Latency and jitter of button-to-blink (Figure 15)

When the system receives inputs at a frequency beyond $1/\delta_r$, the system will not be able to process an input before receiving the next, making δ_r a significant number. As the system falls farther and farther behind wall clock time, its behaviour will degrade into an asynchronous system with none of the temporal behaviour of the underlying Scoria program.

A related metric is the *in-flight input latency*, δ_f : the time it takes for a busy system to respond to external input. When events arrive separated by an amount of time within δ_f , the input queue will be populated with new events by the time the main tick loop thread checks it again, meaning it can resume ticking without putting itself to sleep. δ_f is shorter than δ_r because it eliminates the time spent sleeping and waking. We measured δ_f on our board to be about 45 μ s.

Sparse bursts of events do not overload the system if it can catch up between bursts. The system may even be able to keep up if it is consistently stimulated with a period between δ_r and δ_f . As shown in Table 2, the “button-to-blink” program is able to sustain activity when stimulated by a pulse generator with frequencies of 12 kHz; beyond 12 kHz, the program begins to thrash and drop input events. However, when the delay between successive events is less than δ_r , the computation time per instant

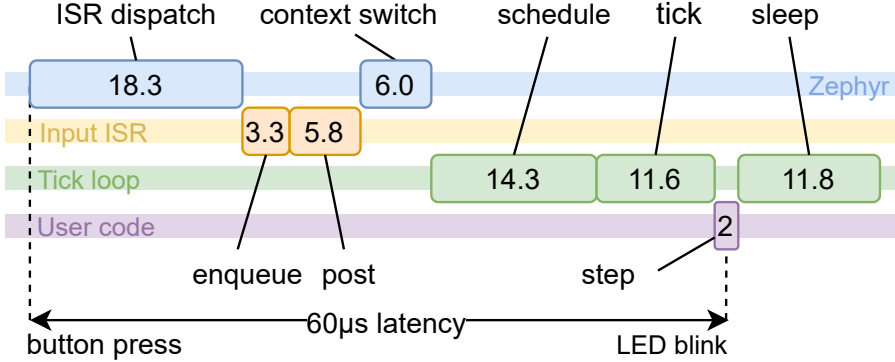


Fig. 16. Timeline in μ s of a single button press of “button-to-blink” (Figure 15), reconstructed from GPIO profiling.

becomes less predictable, even as the system remains responsive. In Table 2, this manifests in increased jitter at input frequencies beyond 8 kHz, at which a square wave has a half-period of 62.5 μ s.

To determine the source of our system’s δ_r of 60 μ s, we profile a single button press of b2b to obtain the timeline in Figure 16. We add code that emits 4-bit codes on GPIO pins for certain events and record them with a logic analyzer. Analyzing the recorded data determines time between events. While this approach is intrusive, we found each GPIO write took only 60 ns, negligible compared to what we are measuring. We find that 24.3 μ s is directly introduced by Zephyr’s own ISR and process scheduling facilities. A further 5.8 μ s is introduced by Zephyr’s semaphore implementation, when we call `sem_post` from the ISR to wake up the tick loop thread. The δ_r of 60 μ s is from Table 2; Figure 16 shows slightly different numbers from a different Zephyr version.

We also measure δ_r for two Zephyr programs written in C to compare with our Scoria implementation. One of our implementations, Figure 2, unrealistically constantly polls for button presses, and represents the lowest theoretical at-rest response time for any Zephyr program. Our other implementation is more realistic, responding to input via an interrupt handler that reports the button press event by placing it in a ring buffer identical to the one in the Scoria runtime and resuming the main thread by releasing a semaphore. The main thread then unblocks, retrieves the event from the ring buffer, and updates the LED. Listing 1 shows this version, which models the Scoria runtime without the SSM “tick” scheduler. We standardized the input event frequency at 2 kHz across all implementations; our results are shown in Table 3.

The Scoria version is the slowest of the three. Figure 16 shows the amount of time spent by Zephyr locating and running the input ISR was roughly 33 μ s. This is comparable to the response time measured by the more realistic Zephyr example

Program	Mean δ_r (μ s)	Min. δ_r (μ s)	Max. δ_r (μ s)
Polling	1.519	0.88	2.17
Interrupt + sem	31.46	31.32	31.59
Scoria	57.96	57.86	58.07

Table 3. The at-rest response time of three button-to-blink implementations fed a 2 kHz input. Data gathered with a logic analyzer sampling at 100 MHz.

Program	Generated Frequency (kHz)	LOC
C	12.2	54
Scoria	6.57	32

Table 4. Implementations’ maximum frequencies and sizes

(Table 3). The overhead imposed by the Scoria runtime system is mainly the parts in green (the *Tick loop* bar), where the Scoria runtime is retrieving the event from the input queue, inserting it in the Scoria environment, and scheduling processes to invoke the Scoria tick loop.

Signal Generator. With these measurements made we can return to the signal generator from Figure 1. We measure the highest frequency (shortest period) it can generate. For the purposes of evaluation, we adapted the example to adjust the half-period `hperiod` in linear increments of 2μ s by the press of a button. Listing 8 shows the complete code. We find that the signal generator from Figure 1 can reliably generate signals with half-period as low as 76μ s, corresponding to a frequency of about 6.6 kHz, with less than 500 ns of jitter—within the level of precision of our oscilloscope. At half-periods between 70μ s– 76μ s, we observe degradation in signal accuracy and consistency: the output signal’s half-period fluctuates between 60.2μ s– 83.4μ s. At lower half-periods, the system ceases to output any signal as it is overwhelmed by the computation load. Once the half-period is increased beyond 76μ s, the system eventually recovers after a burst of high-frequency output, emitted as it catches up with wall-clock time.

Table 4 compares our Scoria implementation with a comparable signal generator written in C (Listing 7). We measure the maximum frequency for each implementation. Like the Scoria runtime, the C implementation populates a ring buffer when the alarm goes off; the main thread is then unblocked by a semaphore release, after which it toggles the LED.

The Zephyr program outperforms the Scoria program, as can be seen in table 4. As soon as the half period becomes lower than the time it takes for the program to handle one wave event, the signal becomes unstable. Since the Scoria runtime adds some overhead when reacting to events, as can be seen in Figure 16, the Scoria program can not handle as low half periods as the Zephyr program. The Zephyr program can generate about twice as high of a frequency.

BLE Frequency Mime. As a slightly more complicated example, we wrote an application where two boards communicate over BLE. One board will advertise varying frequencies using BLE, and the other board will scan for such messages. When a message is intercepted, the board will generate a signal of the detected frequency as it continues to scan for messages.

The C version needs to manage such things as callbacks, clocks, counters, and global state. Any of these areas could hide bugs, and they are all absent from the Scoria version. The Scoria version specifies only application logic, not the details of how it is realized. Listings 3 to 6 show both versions.

8 RELATED WORK

Synchronous Languages. Scoria’s programming model SSM [12] adopts the *synchrony hypothesis*: concurrent execution proceeds as a sequence of synchronized, zero-time instants [1]. While digital hardware designers have used this model for over half a century, French researchers in the 1990s brought the concept to software languages Lustre, Esterel, and Signal [2, 4, 7]. Synchrony simplifies reasoning about concurrent timing behavior by eliminating the non-determinism of asynchrony, and has since featured in languages such as Timber [6] and Lingua Franca [15], and EDSLs like Haski [24] and Copilot [20].

Despite their application in real-time programming, the “French school” of programming languages—Lustre, Esterel, and Signal [2, 4, 7]—do not include time as a first-class language construct. Instead, they specify computation through *successive* instants, and compile to tick functions that must be invoked periodically by a cyclic executive for every instant, even those where no computation takes place. Meanwhile, Scoria’s *after* statement allows users to specify (or compute) concrete time delays, and its sparse execution model does not obligate its runtime to compute inert instants.

Many of the synchronous languages are dataflow-oriented, including Lustre, Signal, and Lingua Franca [2, 7, 15], where programs specify a static, finite dataflow graph where processes (nodes) communicate along fixed channels (edges). In contrast, Scoria is imperative (programs specify control flow); processes are spawned dynamically using *fork*, and the dataflow between them is implicit. Though Esterel is also imperative and includes concurrent control-flow constructs like Scoria’s *fork* and *wait*, it lacks function calls, limiting Esterel programs to a finite number of processes [4].

Like Scoria, Haski [24] and Copilot [20] are deeply embedded Haskell DSLs [22] that generate C code for microcontrollers. Haski is an embedding of Lustre in Haskell for secure IoT programming; Copilot is an EDSL for monitoring software that promises constant-time and -space C code. Both are dataflow languages, so their embedding in Haskell constructs abstract streams instead of Scoria’s syntax trees.

Compiler Testing. While manually written regression test suites remain the standard for compiler development, many have also considered randomly generating and

shrinking compiler test cases [5, 8]. Our work is most similar to RandIR [18], which uses ScalaCheck [17] to test LMS [21], a code generation framework for embedded DSLs. RandIR uses Scala as its test oracle, translating its test candidates to Scala to compare against other language backends; in comparison, Scoria’s test oracle directly interprets generated Scoria IR.

For well-established languages, differential testing can take advantage of multiple compiler implementations, e.g., CSmith for C [25], or multiple optimization levels, e.g., Palka et al. [19] for the Glasgow Haskell Compiler. While Scoria cannot afford such an opportunity as a nascent language, we look forward to adapting our test-driven compiler development to help identify bugs as we begin to implement optimizations that take advantage of Scoria’s synchronous semantics.

Discrete-Event Systems. Ptides[10] is a programming model used to implement discrete-event real-time systems that inspired SSM and Scoria. Unlike Scoria/SSM, however, Ptides implements a *distributed* discrete-event system that cautiously determines when it is safe to process a timestamped message, i.e., when a message with an earlier timestamp cannot appear. We plan to implement distributed Scoria following Ptides.

9 FUTURE WORK

Types. Scoria currently only supports a limited, predefined set of data types. We will add support for more complex types, such as arrays and user-defined algebraic data types (ADTs). Building on McDonnell et al. [16], we can expose a pattern-matching language interface that is familiar to Haskell programmers. We believe ADTs will be a good fit for writing IoT programs; the richness they bring will be especially helpful for developing safe interfaces for peripherals.

Linear Types. *Linear types* [3] were recently added to GHC, which allows certain invariants to be encoded in the types and checked by the compiler. For instance, the compiler can enforce that we schedule our I/O handlers, if `output` and `schedule` are given more precise linear types:

```

1  output :: Int
2      -> (Ref LED -> SSM () %1 -> Compile ())
3      -> Compile ()
4  output i continuation = ...
5
6  schedule :: SSM () %1 -> Compile ()
7  schedule handler = ...

```

`output`’s type indicates that it takes an integer identifying the LED, as well as a *continuation* that takes the LED reference and handler as parameters, where the `%1` indicates that handler must be consumed exactly once. These functions can be invoked as follows:

```

1  program :: Compile ()

```

```
2 program = output 0 $ \ref handler -> do
3   schedule handler -- followed by rest of program setup
```

Since the handler is scheduled inside of the continuation, forgetting to schedule the handler (or scheduling it more than once) is a GHC compile time error. This could otherwise only be checked at code-generation time (host-language run time).

Performance. Experimentally, we identified a few sources of inefficiency that make Scoria significantly slower than counterparts implemented directly in C and Zephyr. While the C and Zephyr versions process an event as soon as the associated interrupt handler is invoked, Scoria has to enqueue the message and wake up the main tick-thread that retrieves it and actually runs the program, which add overhead.

The benchmarked programs can perform significantly better if the required OS facilities were developed directly on the bare metal rather than acquired by Zephyr. While Zephyr gives quick access to many convenient abstractions, they are developed in a very general way to be suitable for many kind of applications. If we could tailor these abstractions to just the needs of Scoria, we might be able to get some speedup. We will try this, as well as trying out other operating systems.

10 CONCLUSIONS

While the Scoria language is still a simple language, Scoria programs are short and clutter-free, and the compiler generates resource-efficient code. Implementing Scoria as an EDSL has led to several advantages for language and compiler development. Taking advantage of features and tooling provided by the host language, we can rapidly prototype our language. Using primitives to derive new operations such as delay is straightforward and transparent. Since the EDSL itself is simply a Haskell library whose API constructs an AST, we can easily experiment with exposing and modifying compiler internals with little coordination with the language front-end.

We were able to confidently iterate on our compiler due to our Scoria interpreter, which we developed early on. Using the interpreter as a language oracle, we used QuickCheck to thoroughly and efficiently test corner cases in our language, identifying several bugs that would have otherwise taken much longer to identify. To help us better understand what went wrong, our shrinker produced minimal failing test programs which easily pointed to the source of each bug we found.

We extended the RTS proposed by Edwards & Hui [12] with Zephyr OS bindings to run Scoria programs on real hardware. I/O peripherals are exposed via Scoria references so that Scoria programs can converse with peripherals natively, avoiding the callback hell typical of IoT applications. This interaction model has allowed us to easily add support for a handful of peripherals, including GPIO and BLE. Scoria allows users to precisely specify the priority of I/O handling code relative to concurrent user code and make application-specific reactivity and sensitivity trade-offs. Our stress tests confirm we can write real-time applications with robust overflow behaviour.

While Scoria remains an experimental language, our successes with it to date have illustrated how implementing an EDSL provides a faster route to language

implementation for experimental languages. As expected, Scoria has raised many questions about how any sparse synchronous language should be designed, notably how its interface with the outside world should be specified, and given us a quick route to experimenting with solutions to these problems. Work on Scoria continues and will, at the very least, inform language design in similar domains.

ACKNOWLEDGMENTS

Hui and Edwards were supported in part by the National Institutes of Health (NIH) under grant 1RF1MH120034-01.

Krook and Claessen were supported by the Swedish Foundation for Strategic Research (SSF) under the project Octopi (Ref. RIT17-0023) and by the Swedish research agency Vetenskapsrådet under the project SyTeC (Ref. 2016-06204).

An additional thanks goes to Agustín Mista for help writing the plugin described in this paper.

REFERENCES

- [1] Albert Benveniste, Paul Caspi, Stephen A. Edwards, Nicolas Halbwachs, Paul Le Guernic, and Robert de Simone. 2003. The Synchronous Languages 12 Years Later. *Proc. IEEE* 91, 1 (jan 2003), 64–83.
- [2] Albert Benveniste, Paul Le Guernic, and Christian Jacquemot. 1991. Synchronous programming with events and relations: the SIGNAL language and its semantics. *Science of computer programming* 16, 2 (1991), 103–149.
- [3] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: practical linearity in a higher-order polymorphic language. *Proceedings of the ACM on Programming Languages* 2, POPL (jan 2018), 1–29. <https://doi.org/10.1145/3158093>
- [4] Gérard Berry and Georges Gonthier. 1992. The Esterel synchronous programming language: Design, semantics, implementation. *Science of computer programming* 19, 2 (1992), 87–152.
- [5] A.S. Boujarwah and K. Saleh. 1997. Compiler test case generation methods: a survey and assessment. *Information and Software Technology* 39, 9 (1997), 617–625. [https://doi.org/10.1016/S0950-5849\(97\)00017-7](https://doi.org/10.1016/S0950-5849(97)00017-7)
- [6] Magnus Carlsson, Johan Nordlander, and Dick Kieburtz. 2003. The Semantic Layers of Timber. In *Asian Symposium on Programming Languages and Systems*. 339–356. https://doi.org/10.1007/978-3-540-40018-9_22
- [7] P. Caspi, D. Pilaud, N. Halbwachs, and J. A. Plaice. 1987. LUSTRE: A Declarative Language for Real-Time Programming. In *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages* (Munich, West Germany) (POPL '87). Association for Computing Machinery, New York, NY, USA, 178–188. <https://doi.org/10.1145/41625.41641>
- [8] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A Survey of Compiler Testing. *Comput. Surveys* 53, 1 (may 2020), 1–36. <https://doi.org/10.1145/3363562>
- [9] Koen Claessen and John Hughes. 2000. QuickCheck: a lightweight tool for random testing of Haskell programs, In *Proceedings of the fifth ACM SIGPLAN international conference on Functional programming*. *SIGPLAN Not.* 35, 9, 268–279. <https://doi.org/10.1145/357766.351266>
- [10] Patricia Derler, Thomas H Feng, Edward A Lee, Slobodan Matic, Hiren D Patel, Yang Zheo, and Jia Zou. 2008. *PTIDES: A programming model for distributed real-time embedded systems*. Technical Report. CALIFORNIA UNIV BERKELEY DEPT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE.
- [11] Adam Dunkels, Oliver Schmidt, Thiemo Voigt, and Muneeb Ali. 2006. Protothreads: Simplifying Event-Driven Programming of Memory-Constrained Embedded Systems. In *Proceedings of the 4th International Conference on Embedded Networked Sensor Systems* (Boulder, Colorado, USA) (*SenSys '06*). Association for Computing Machinery, New York, NY, USA, 29–42. <https://doi.org/10.1145/1182807.1182811>
- [12] Stephen A. Edwards and John Hui. 2020. The Sparse Synchronous Model. In *Forum on Specification and Design Languages (FDL)*. IEEE, Kiel, Germany, 1–8. <https://doi.org/10.1109/FDL50818.2020.9232938>
- [13] Cordelia V Hall, Kevin Hammond, Simon L Peyton Jones, and Philip L Wadler. 1996. Type classes in Haskell. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 18, 2 (1996), 109–138.
- [14] Paul Hudak. 1996. Building Domain-Specific Embedded Languages. *Comput. Surveys* 28, 4es (dec 1996), 196–es. <https://doi.org/10.1145/242224.242477>
- [15] Marten Lohstroh, Christian Menard, Soroush Bateni, and Edward A. Lee. 2021. Toward a Lingua Franca for Deterministic Concurrent Systems. *ACM Transactions on Embedded Computing Systems* 20, 4 (July 2021), 1–27. <https://doi.org/10.1145/3448128>
- [16] Trevor L. McDonell, Joshua D. Meredith, and Gabriele Keller. 2021. Embedded Pattern Matching. *arXiv preprint arXiv:2108.13114* (2021). [arXiv:2108.13114 \[cs.PL\]](https://arxiv.org/abs/2108.13114)
- [17] R. Nilsson. 2014. *ScalaCheck: The Definitive Guide*. Artima Press. <https://books.google.se/books?id=AOvcoQEACAAJ>
- [18] Georg Ofenbeck, Tiark Rumpf, and Markus Püschel. 2016. RandIR: differential testing for embedded compilers. In *Proceedings of the 2016 7th ACM SIGPLAN Symposium on Scala*. ACM. <https://doi.org/>

- 10.1145/2998392.2998397
- [19] Michal H. Palka, Koen Claessen, Alejandro Russo, and John Hughes. 2011. Testing an Optimising Compiler by Generating Random Lambda Terms. In *Proceedings of the 6th International Workshop on Automation of Software Test* (Waikiki, Honolulu, HI, USA) (*AST '11*). Association for Computing Machinery, New York, NY, USA, 91–97. <https://doi.org/10.1145/1982595.1982615>
 - [20] Lee Pike, Alwyn Goodloe, Robin Morisset, and Sebastian Niller. 2010. Copilot: A hard real-time runtime monitor. In *International Conference on Runtime Verification*. Springer, Springer Berlin Heidelberg, Berlin, Heidelberg, 345–359.
 - [21] Tiark Rompf and Martin Odersky. 2012. Lightweight modular staging: a pragmatic approach to runtime code generation and compiled DSLs. *Commun. ACM* 55, 6 (jun 2012), 121–130. <https://doi.org/10.1145/2184319.2184345>
 - [22] Josef Svenningsson and Emil Axelsson. 2013. Combining Deep and Shallow Embedding for EDSL. In *Trends in Functional Programming*, Hans-Wolfgang Loidl and Ricardo Peña (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 21–36.
 - [23] The Zephyr Project. 2021. The Zephyr Project. <https://www.zephyrproject.org/>.
 - [24] Nachiappan Valliappan, Robert Krook, Alejandro Russo, and Koen Claessen. 2020. Towards Secure IoT Programming in Haskell. In *Proceedings of the 13th ACM SIGPLAN International Symposium on Haskell* (Virtual Event, USA) (*Haskell 2020*). Association for Computing Machinery, New York, NY, USA, 136–150. <https://doi.org/10.1145/3406088.3409027>
 - [25] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Jose, California, USA) (*PLDI '11*). Association for Computing Machinery, New York, NY, USA, 283–294. <https://doi.org/10.1145/1993498.1993532>

```
1  {-# LANGUAGE ImplicitParams #-}
2  {-# OPTIONS_GHC -fplugin=SSM.Plugin
3      -fplugin-opt=SSM.Plugin:mode=routine #-}
4  module Blinky where
5
6  import SSM.Language
7  import SSM.Frontend.Peripheral.LED
8
9  program :: (?pin :: Ref GPIO) => SSM ()
10 program = routine $ while true $ do
11     after (msecs 1000) ?pin (not $ deref ?pin)
12     wait ?pin
13
14 main :: Compile ()
15 main = do
16     (pin, handler) <- output 1
17     let ?pin = pin
18     schedule handler
19     schedule program
```

Fig. 1. The “blinky” example from Listing 2, implemented in Scoria.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3
4  static const struct gpio_dt_spec button =
5      GPIO_DT_SPEC_GET_OR(DT_ALIAS(sw0), gpios, {0});
6
7  static struct gpio_dt_spec led =
8      GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
9
10 void main() {
11     gpio_pin_configure_dt(&button, GPIO_INPUT);
12     gpio_pin_configure_dt(&led, GPIO_OUTPUT);
13
14     while (1)
15         gpio_pin_set_dt(&led, gpio_pin_get_dt(&button));
16 }
```

Fig. 2. The code of the polling button-to-blink code. 100% of the CPU is allocated to reflecting the button state on the LED, which gives a kind of theoretical minimum for how fast the response time *could* be.

Listing 1. The button-to-blink program implemented using Zephyr, where input events are delivered to an input ISR (line 14). When an event arrives, it is placed in a ring buffer (line 17-20), and the main thread is woken up by releasing a semaphore (line 22). The main thread retrieves the event from the ring buffer (line 39-40) and toggles the LED.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3  #include "rb.h"
4
5  K_SEM_DEFINE(my_sem, 0, 1);
6  SSM_RB_DEFINE(int, ssm_input_buffer, 12);
7
8  static const struct gpio_dt_spec button =
9      GPIO_DT_SPEC_GET_OR(DT_ALIAS(sw0), gpios, {0});
10 static struct gpio_dt_spec led =
11     GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
12 static struct gpio_callback button_cb_data;
13
14 void button_pressed(const struct device *dev
15                     , struct gpio_callback *cb
16                     , uint32_t pins) {
17     int *msg = ssm_rb_writer_alloc(ssm_input_buffer);
```

Creating a Language for Writing Real-Time Applications for the Internet of Things

```
18     if(msg) {
19         *msg = gpio_pin_get_dt(&button);
20         ssm_rb_writer_commit(ssm_input_buffer);
21     }
22     k_sem_give(&my_sem);
23 }
24
25 void main(void)
26 {
27     gpio_pin_configure_dt(&button, GPIO_INPUT);
28     gpio_pin_interrupt_configure_dt(&button, GPIO_INT_EDGE_BOTH);
29     gpio_init_callback(&button_cb_data
30                       , button_pressed
31                       , BIT(button.pin));
32     gpio_add_callback(button.port, &button_cb_data);
33
34     gpio_pin_configure_dt(&led, GPIO_OUTPUT);
35
36     while(true) {
37         k_sem_take(&my_sem, K_FOREVER);
38         int *msg = ssm_rb_reader_claim(ssm_input_buffer);
39         if(msg) {
40             gpio_pin_set_dt(&led, *msg);
41             ssm_rb_reader_free(ssm_input_buffer);
42         }
43     }
44 }
```

Listing 2. “Blinky” implementation with correct temporal behavior, implemented using Zephyr RTOS. To achieve more precise timing, the code configures the NRF52840-DK board to use its external crystal oscillator as its clock source. This implementation does not suffer from drift due to its use of logical time, though this comes at the complexity of (1) maintaining the value of the logical clock, and (2) decomposing the application into interrupt handlers.

```
1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3  #include <drivers/counter.h>
4  #include <drivers/clock_control.h>
5  #include <drivers/clock_control/nrf_clock_control.h>
6  static struct gpio_dt_spec led =
7      GPIO_DT_SPEC_GET_OR(DT_ALIAS(ssm_led), gpios, {0});
8  const struct device *timer_dev, *clock_dev;
9  struct counter_alarm_cfg alarm_cfg;
```

```

10  uint32_t current; // Logical time
11
12  int configure_led(void) {
13      if (led.port && !device_is_ready(led.port))
14          return -ENODEV;
15      gpio_pin_configure(&led, GPIO_OUTPUT);
16  }
17
18  int configure_timer(void) {
19      timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
20      if (!timer_dev)
21          return -ENODEV;
22      counter_start(timer_dev);
23
24      clock_dev = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
25      if (!clock_dev)
26          return -ENODEV;
27      clock_control_on(clock_dev, CLOCK_CONTROL_NRF_SUBSYS_HF);
28  }
29
30  void set_alarm(uint64_t delay_in_us) {
31      alarm_cfg.ticks = (current += counter_us_to_ticks(timer_dev, delay_in_us));
32      counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
33  }
34
35  void counter_interrupt_fn(const struct device *counter_dev
36                          , uint8_t chan_id
37                          , uint32_t ticks, void *user_data) {
38      gpio_pin_toggle_dt(&led);
39      set_alarm(1000000); // 1000000 us = 1000 ms
40  }
41
42  int configure_alarm(void) {
43      alarm_cfg.flags = COUNTER_ALARM_CFG_ABSOLUTE |
44                      COUNTER_ALARM_CFG_EXPIRE_WHEN_LATE;
45      alarm_cfg.callback = counter_interrupt_fn;
46      alarm_cfg.user_data = &alarm_cfg;
47
48      counter_get_value(timer_dev, &current);
49  }
50
51  void main(void) {
52      configure_led();

```

```

53     configure_timer();
54     configure_alarm();
55     set_alarm(1000000); // 1000000 us = 1000 ms
56 }

```

Listing 3. The C version of the BLE mime broadcaster. This application broadcasts one of two frequencies, switching between the two every five seconds.

```

1  #include <zephyr.h>
2  #include <device.h>
3  #include <drivers/gpio.h>
4  #include <drivers/counter.h>
5  #include <drivers/clock_control.h>
6  #include <drivers/clock_control/nrf_clock_control.h>
7  #include <bluetooth/bluetooth.h>
8  #include <bluetooth/hci.h>
9  static void counter_interrupt_fn( const struct device *counter_dev
10                                  , uint8_t chan_id
11                                  , uint32_t ticks, void *user_data);
12
13 int ssm_timer_configure_clock(void) {
14     const struct device *clock;
15     clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
16     clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
17     return 0;
18 }
19
20 const struct device *timer_dev;
21 struct counter_alarm_cfg alarm_cfg;
22
23 void set_alarm(uint64_t delay_in_us) {
24     alarm_cfg.ticks = counter_us_to_ticks(timer_dev, delay_in_us);
25     counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
26 }
27
28 uint64_t broadcast_data = 0;
29 static struct bt_data ad[] = {
30     BT_DATA(BT_DATA_MANUFACTURER_DATA, &broadcast_data, 8));
31 K_THREAD_STACK_DEFINE(my_stack_area, 512);
32 struct k_work_q my_work_q;
33 #define SHORT_FREQ 200000
34 #define LONG_FREQ 500000
35

```

```

36 void alternate_frequency_broadcast(struct k_work *item) {
37     bt_le_adv_stop();
38     broadcast_data = broadcast_data == SHORT_FREQ ? LONG_FREQ : SHORT_FREQ;
39     bt_le_adv_start(BT_LE_ADV_NCONN_IDENTITY, ad, ARRAY_SIZE(ad), NULL, 0);
40 }
41
42 K_WORK_DEFINE(my_work_item, alternate_frequency_broadcast);
43
44 static void counter_interrupt_fn( const struct device *counter_dev
45                                 , uint8_t chan_id
46                                 , uint32_t ticks, void *user_data) {
47     k_work_submit(&my_work_item);
48     set_alarm(5000000);
49 }
50
51 void main() {
52     bt_enable(NULL);
53     k_work_queue_start(&my_work_q, my_stack_area
54                       , K_THREAD_STACK_SIZEOF(my_stack_area), 5, NULL);
55     timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
56     counter_start(timer_dev);
57     ssm_timer_configure_clock();
58     alarm_cfg.flags = 0;
59     alarm_cfg.callback = counter_interrupt_fn;
60     alarm_cfg.user_data = &alarm_cfg;
61     set_alarm(5000000);
62 }

```

Listing 4. The C version of the BLE mime generator. This application scans for broadcasted messages, and generates a frequency on LED 0. If a broadcasted message is detected, the generated frequency is the one specified in the advertisement payload.

```

1  #include <zephyr.h>
2  #include <device.h>
3  #include <drivers/gpio.h>
4  #include <drivers/counter.h>
5  #include <drivers/clock_control.h>
6  #include <drivers/clock_control/nrf_clock_control.h>
7  #include <bluetooth/bluetooth.h>
8  #include <bluetooth/hci.h>
9  static struct gpio_dt_spec led =
10     GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
11 static void counter_interrupt_fn( const struct device *counter_dev

```


Creating a Language for Writing Real-Time Applications for the Internet of Things

```
12         , uint8_t chan_id
13         , uint32_t ticks, void *user_data);
14
15 int ssm_timer_configure_clock(void) {
16     const struct device *clock;
17     clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
18     clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
19     return 0;
20 }
21 const struct device *timer_dev;
22 struct counter_alarm_cfg alarm_cfg;
23
24 void set_alarm(uint64_t delay_in_us) {
25     alarm_cfg.ticks = counter_us_to_ticks(timer_dev, delay_in_us);
26     counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
27 }
28
29 uint64_t period_in_us = 1000000;
30 uint64_t next_value = 0;
31
32 static void counter_interrupt_fn( const struct device *counter_dev
33                                 , uint8_t chan_id
34                                 , uint32_t ticks, void *user_data) {
35     gpio_pin_set_dt(&led, next_value);
36     next_value = next_value ^ 1UL;
37     set_alarm(period_in_us);
38 }
39
40 static void device_found( const bt_addr_le_t *addr, int8_t rssi
41                          , uint8_t type
42                          , struct net_buf_simple *ad) {
43     if (type != BT_GAP_ADV_TYPE_ADV_NONCONN_IND) return;
44     if (ad->data[0] == 9 && ad->data[1] == BT_DATA_MANUFACTURER_DATA) {
45         uint64_t res = 0;
46         for (int i = 7; i >= 0; i--) { res = (res << 8) |
47                                         (uint64_t)ad->data[2 + i]; }
48         period_in_us = res;
49     }
50 }
51
52 void main() {
53     bt_enable(NULL);
54     timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
```

```

55     counter_start(timer_dev);
56     ssm_timer_configure_clock();
57     alarm_cfg.flags = 0;
58     alarm_cfg.callback = counter_interrupt_fn;
59     alarm_cfg.user_data = &alarm_cfg;
60     bt_le_scan_start(BT_LE_SCAN_PASSIVE, device_found);
61     gpio_pin_configure_dt(&led, GPIO_OUTPUT);
62     set_alarm(period_in_us);
63 }

```

Listing 5. The Scoria version of the BLE mime broadcaster. It implements the same application as the one in Listing 3. See Section 5 for a description of what delay does.

```

1  {-# LANGUAGE ImplicitParams #-}
2  {-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
3  module FrequencyMime where
4
5  import SSM.Language
6  import SSM.Frontend.Peripheral.BasicBLE
7
8  freqCount :: Ref Time -> SSM () -- concurrent process dictating which
9                                   -- frequency to broadcast
10 freqCount period = routine $ while true $ do
11     period <~ (msecs 200)
12     delay (secs 5)
13     period <~ (msecs 500)
14     delay (secs 5)
15
16 broadcastCount :: (?ble :: BBLE) => Ref Time -> SSM ()
17 broadcastCount count = routine $ while true $ do
18     enableBroadcast $ time2ns $ deref count
19     delay (secs 5)
20     disableBroadcast
21
22 counterEntry :: (?ble :: BBLE) => SSM ()
23 counterEntry = routine $ do
24     count <- var $ secs 1 -- create the reference that is shared between the
25                             -- concurrent processes
26     fork [ freqCount count, broadcastCount count ]
27
28 counter :: Compile ()
29 counter = do
30     (ble, broadcast, scanning) <- enableBasicBLE

```

```

31
32     let ?ble = ble
33
34     schedule counterEntry
35     schedule broadcast
36     schedule scanning

```

Listing 6. The Scoria version of the BLE mime generator. It implements the same application as the one in Listing 4. See Section 5 for a description of what delay does.

```

1  {-# LANGUAGE ImplicitParams #-}
2  {-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
3  module FrequencyMime where
4
5  import SSM.Language
6  import SSM.Frontend.Peripheral.LED
7  import SSM.Frontend.Peripheral.BasicBLE
8
9  -- scan for broadcasted frequencies and write them to the shared reference
10 bleHandler :: (?ble :: BBLE) => Ref Time -> SSM ()
11 bleHandler period = routine $ do
12     enableScan
13     while true $ do
14         wait scanref
15         period <~ nsecs (deref scanref)
16         delay (secs 5)
17
18 -- generate a frequency with a half period dictated by the value of
19 -- the shared reference
20 freqGen :: (?led0::Ref LED) => Ref Time -> SSM ()
21 freqGen period = routine $ while true $ do
22     after (deref period) ?led0 (not' $ deref ?led0)
23     wait ?led0
24
25 entry :: (?ble :: BBLE, ?led0::Ref LED) => SSM ()
26 entry = routine $ do
27     period <- var $ secs 1 -- create reference that is shared between the
28                             -- concurrent processes
29     fork [freqGen period, bleHandler period]
30
31 generator :: Compile ()
32 generator = do
33     (led, handler) <- output 0

```

```

34  (ble, broadcast, scanning) <- enableBasicBLE
35
36  let ?led0 = led
37      ?ble = ble
38
39  schedule handler
40  schedule entry
41  schedule broadcast
42  schedule scanning

```

Listing 7. The frequency generator implemented using Zephyr. When the timer goes off, the associated event (the new state of the LED) is placed in the ring buffer (line 38-41), and the main thread is woken up by releasing a semaphore (line 43). The main thread retrieves the event (line 63-66), and schedules the next wake-up point (line 67).

```

1  #include <zephyr.h>
2  #include <drivers/gpio.h>
3  #include <drivers/counter.h>
4  #include <drivers/clock_control.h>
5  #include <drivers/clock_control/nrf_clock_control.h>
6  #include "rb.h"
7  K_SEM_DEFINE(my_sem, 0, 1);
8  SSM_RB_DEFINE(uint8_t, ssm_input_buffer, 12);
9  static struct gpio_dt_spec led =
10      GPIO_DT_SPEC_GET_OR(DT_ALIAS(led0), gpios, {0});
11  static void counter_interrupt_fn( const struct device *counter_dev
12                                  , uint8_t chan_id
13                                  , uint32_t ticks, void *user_data);
14  uint32_t current;
15  const struct device *timer_dev;
16  struct counter_alarm_cfg alarm_cfg;
17  int ssm_timer_configure_clock(void) {
18      const struct device *clock;
19      clock = device_get_binding(DT_LABEL(DT_INST(0, nordic_nrf_clock)));
20      clock_control_on(clock, CLOCK_CONTROL_NRF_SUBSYS_HF);
21      return 0;
22  }
23
24  void set_alarm(uint64_t delay_in_us) {
25      alarm_cfg.ticks = (current += counter_us_to_ticks(timer_dev, delay_in_us));
26      counter_set_channel_alarm(timer_dev, 0, &alarm_cfg);
27  }
28  uint8_t next;

```

```

29
30 static void counter_interrupt_fn( const struct device *counter_dev
31                                , uint8_t chan_id
32                                , uint32_t ticks, void *user_data) {
33     next = next ? 0 : 1;
34     uint8_t *msg = ssm_rb_writer_alloc(ssm_input_buffer);
35     if(msg) { *msg = next; ssm_rb_writer_commit(ssm_input_buffer); }
36     k_sem_give(&my_sem);
37 }
38
39 void main() {
40     gpio_pin_configure_dt(&led, GPIO_OUTPUT);
41     timer_dev = device_get_binding(DT_LABEL(DT_ALIAS(ssm_timer)));
42     counter_start(timer_dev);
43     ssm_timer_configure_clock();
44     counter_get_value(timer_dev, &current);
45     alarm_cfg.flags = COUNTER_ALARM_CFG_ABSOLUTE |
46                     COUNTER_ALARM_CFG_EXPIRE_WHEN_LATE;
47     alarm_cfg.callback = counter_interrupt_fn;
48     alarm_cfg.user_data = &alarm_cfg;
49     uint32_t half_period = 41;
50     set_alarm(half_period);
51     while(true) {
52         k_sem_take(&my_sem, K_FOREVER);
53         uint8_t *msg = ssm_rb_reader_claim(ssm_input_buffer);
54         if(msg) {
55             gpio_pin_set_dt(&led, *msg);
56             ssm_rb_reader_free(ssm_input_buffer);
57             set_alarm(half_period);
58         }
59     }
60 }

```

Listing 8. A frequency generator implemented in Scoria. The half period of the generated frequency is adjusted in steps of 2 μ s when button 0 or button 1 is pressed.

```

1 {-# LANGUAGE ImplicitParams #-}
2 {-# LANGUAGE FlexibleContexts #-}
3 {-# LANGUAGE RebindableSyntax #-}
4 {-# OPTIONS_GHC -fplugin=SSM.Plugin -fplugin-opt=SSM.Plugin:mode=routine #-}
5 module Generator where
6
7 import Prelude

```

```

8  import SSM.Language
9  import SSM.Frontend.Peripheral.GPIO
10
11  -- signal generator, generating a signal on ?out0 with a half period
12  -- specified by hpr
13  sig_gen :: (?out0 :: Ref GPIO) => Ref Time -> SSM ()
14  sig_gen hpr = routine $ while true $ do
15      after (deref hpr) ?out0 (not' $ deref ?out0)
16      wait ?out0
17
18  -- increments or decrements the half period by 2 us, depending on which of
19  -- two buttons was pressed
20  button_handler :: (?b1 :: Ref Switch, ?b2 :: Ref Switch) => Ref Time -> SSM ()
21  button_handler hpr = routine $ while true $ do
22      wait (?b1, ?b2)
23      if changed ?b1 -- here we woke up, so either b1 or b2 was pressed
24          then hpr <- deref hpr + usecs 2
25          else hpr <- deref hpr - usecs 2
26
27  -- create shared half period reference and initialize it to 1 second, before
28  -- spawning the concurrent child processes
29  entry :: (?out0 :: Ref GPIO, ?b1 :: Ref Switch, ?b2 :: Ref Switch) => SSM ()
30  entry = routine $ do
31      hp <- var $ usecs 1000000
32      fork [ sig_gen hp, button_handler hp]
33
34  -- Specify what inputs and outputs the program uses, and specify the initial
35  -- contents on the ready queue
36  main :: Compile ()
37  main = do
38      (led, handler) <- output 0
39      button1 <- input 0
40      button2 <- input 1
41
42      let ?out0 = led
43          ?b1 = button1
44          ?b2 = button2
45
46      schedule handler
47      schedule entry

```

Listing 9. The generated C code from the Scoria program in Figure 15. For more details of code such as this, see [12].

Creating a Language for Writing Real-Time Applications for the Internet of Things

```
1  #include "ssm-platform.h"
2  #ifndef SSM_DEBUG_TRACE
3  #define SSM_DEBUG_TRACE(...) do ; while (0)
4  #endif
5  #ifndef SSM_DEBUG_MICROTICK
6  #define SSM_DEBUG_MICROTICK(...) do ; while (0)
7  #endif
8  ssm_bool_t input0;
9  ssm_bool_t output1;
10 typedef char event;
11 typedef struct {
12     struct ssm_act act;
13     struct ssm_trigger trig1;
14 } act_b2b_t;
15 struct ssm_act *enter_b2b( struct ssm_act *caller, ssm_priority_t priority
16                           , ssm_depth_t depth);
17 void step_b2b(struct ssm_act *actg);
18 struct ssm_act *enter_b2b( struct ssm_act *caller, ssm_priority_t priority
19                           , ssm_depth_t depth) {
20     struct ssm_act *actg =
21         ssm_enter(sizeof(act_b2b_t), step_b2b, caller, priority, depth);
22     act_b2b_t *acts = container_of(actg, act_b2b_t, act);
23     acts->trig1.act = actg;
24     return actg;
25 }
26 void step_b2b(struct ssm_act *actg) {
27     act_b2b_t *acts = container_of(actg, act_b2b_t, act);
28     SSM_DEBUG_TRACE("ActStepBegin \"b2b\"");
29     SSM_DEBUG_MICROTICK();
30     switch (actg->pc) {
31     case 0:;
32         while (true) {
33             SSM_DEBUG_MICROTICK();
34             SSM_DEBUG_TRACE("ActSensitize \"input0\"");
35             ssm_sensitize(&(&input0)->sv, &acts->trig1);
36             actg->pc = 1;
37             return;
38         case 1:;
39             ssm_desensitize(&acts->trig1);
40             ssm_assign_bool(&output1, actg->priority, (&input0)->value);
41         }
42     default: break;
43 }
```

```
44     ssm_leave(actg, sizeof(act_b2b_t));
45 }
46 int ssm_program_initialize(void) {
47     ssm_initialize_bool(&input0);
48     ssm_assign_bool(&input0, 0, 0);
49     ssm_initialize_bool(&output1);
50     ssm_assign_bool(&output1, 0, 0);
51     bind_static_input_device((ssm_sv_t *)&input0.sv, 0U);
52     ssm_activate(enter_b2b(&ssm_top_parent
53         , SSM_ROOT_PRIORITY + 0 * (1 << SSM_ROOT_DEPTH - 1), SSM_ROOT_DEPTH - 1));
54     ssm_activate(bind_static_output_device(&ssm_top_parent
55         , SSM_ROOT_PRIORITY + 1 * (1 << SSM_ROOT_DEPTH - 1)
56         , SSM_ROOT_DEPTH - 1, &output1.sv, 0U));
57     return 0;
58 }
```